MiniJinja

Render Jinja-style templates directly from SQL

Install & load

```
INSTALL minijinja FROM community;
LOAD minijinja;
```

01 At a glance

Turn JSON context into rendered text using familiar Jinja variables, filters, loops, and conditions. Generate one HTML fragment, message, or configuration block per row while keeping the source data and rendering query together.

Template → JSON context → Rendered text

02 SQL entry points + template syntax

`minijinja_render(template, *, ...)`

Render a template with no context; options by name.

`minijinja_render_with_context`

Pass template and per-row JSON context, then named options.

`autoescape := false`

Disable HTML escaping for plain text and other non-HTML output.

`{{ value | upper }}`

Interpolate a value and apply a filter.

`{% for x in items %}`

Loop through context items; finish with endfor.

`{% if condition %}`

Choose content using if / elif / else / endif.

03 Keep in mind

Choose the output format

HTML autoescape is enabled by default: B&O renders as B&O. Keep it for HTML, or pass autoescape := false for plain text. Escaping for SQL, JSON, and shell syntax is a separate concern.

04 Render a message per row

```
SELECT minijinja_render_with_context(
  'Hello {{ name | upper }}!',
  json_object('name', name)
) AS message
FROM (VALUES ('Ada'), ('Grace')) t(name);
-- Hello ADA! / Hello GRACE!
```

Each row supplies its own context to the same template.

05 Loop through a list

```
SELECT minijinja_render_with_context(
  '{% for x in xs %}{{ x }}' ||
  '{% if not loop.last %}, {% endif %}' ||
  '{% endfor %}',
  '{"xs":["A","B","C"]}'
);
-- A, B, C
```

loop.last makes separators conditional without a trailing comma.

06 Render unescaped plain text

```
SELECT minijinja_render_with_context(
  '{{ v }}', '{"v":"B&O"}',
  autoescape := false
);
-- B&O
```

Omit the option for HTML output: B&O.

07 In practice

Keep values in the context

Use minijinja_render_with_context for JSON built from row columns. Keep templates trusted. If generating SQL, bind untrusted data through prepared statements; template interpolation is not a substitute for SQL parameters.

Full docs & examples ↗ Source ↗